Design Pattern Selection for VOCO

Why Strategy, Adapter, and Factory Method were selected — and why they are specifically suitable for ForgotPasswordController

1. Objective

The purpose of this document is to justify the selection of three design patterns for the VOCO MERN project. The selection is based on the design-pattern categories and definitions presented in the Unit 4 - Part 1 - Design Pattern material.

The selected patterns are **Strategy**, **Adapter**, and **Factory Method**. Together, they represent the three major GoF categories covered in the material: Behavioral, Structural, and Creational.

2. Selected Patterns at a Glance

Pattern	Category
Strategy	Behavioral
Adapter	Structural
Factory Method	Creational

3. Why These Three Patterns?

Strategy — Behavioral: The PPT defines Strategy as a pattern that “Encapsulates an algorithm inside a class.” This makes it appropriate for separating password-validation rules or other interchangeable business-rule algorithms.

Adapter — Structural: The PPT defines Adapter as a pattern that “Match interfaces of different classes.” This makes it appropriate for adapting an existing or external notification/email service to an interface expected by VOCO.

Factory Method — Creational: The PPT defines Factory Method as a pattern that “Creates an instance of several derived classes.” This makes it appropriate for separating service-object creation from the controller.

4. Why Specifically ForgotPasswordController?

ForgotPasswordController is a practical location because the password-recovery workflow contains several separate responsibilities. The controller deals with OTP generation and verification, password-reset input validation, password processing, user lookup, and sending notification email.

These responsibilities provide clear places where the three selected patterns can be introduced without redesigning the entire VOCO application. The intention is to keep the implementation small and understandable rather than adding patterns only for the sake of having more patterns.

5. Strategy Pattern — Password Validation

Why it fits: The password-reset operation contains validation rules such as checking the minimum password length and confirming that the new password matches the confirmation password. These rules can be separated into a validation strategy.

Suggested structure:
ForgotPasswordController → PasswordValidationStrategy → Validation Implementation

Benefit for VOCO: The controller coordinates the reset process while the strategy handles the validation algorithm. If the validation rules change or another validation approach is introduced, the strategy can change without placing all validation logic inside the controller.

6. Adapter Pattern — Email / Notification Integration

Why it fits: The forgot-password workflow sends OTP and password-reset notifications through an existing mail service. An Adapter can provide a VOCO-specific interface around that service.

Suggested structure:
ForgotPasswordController → NotificationAdapter → Existing Mail Service

Benefit for VOCO: The controller does not need to depend directly on the particular interface of the mail implementation. The adapter acts as the compatibility layer between the application's expected interface and the service interface.

7. Factory Method — Notification / Service Creation

Why it fits: Factory Method can move service-object creation or selection out of the controller. For example, a notification factory can create or select the notification service used by the password-recovery workflow.

Suggested structure:
ForgotPasswordController → NotificationFactory → Notification Service

Benefit for VOCO: Object-creation logic is kept in one place. If another notification implementation is introduced later, the factory can handle the selection or creation rather than placing that responsibility throughout the controller.

8. How the Three Patterns Work Together

ForgotPasswordController
Coordinates the password-recovery workflow
↓
Strategy
Handles password-validation algorithm/rules
↓
Factory Method
Creates/selects the required service object
↓
Adapter
Connects the application interface to the mail/notification service

9. Why Not Use More Patterns?

The goal is to implement three meaningful patterns rather than introduce unnecessary abstraction. The course material notes that design patterns can add complexity and may be unnecessary for small or simple applications. Therefore, using three patterns that correspond to identifiable responsibilities in the existing workflow provides a clearer and more defensible design.

10. Final Selection and Justification

Pattern	Applied To	Reason for Selection
Strategy	Password validation	Separates validation/business-rule algorithms from the controller.
Adapter	Email / notification integration	Adapts the existing service interface to the application's expected interface.
Factory Method	Service-object creation	Separates creation/selection of service objects from controller logic.

Conclusion: Strategy, Adapter, and Factory Method provide a balanced selection for VOCO. They cover Behavioral, Structural, and Creational design concerns while fitting concrete responsibilities around the ForgotPasswordController. The selection is therefore based on where the patterns can solve recognizable design problems, not simply on the number of patterns implemented.

Source basis: Unit 4 - Part 1 - Design Pattern, uploaded course material. The document uses the terminology and definitions presented in that material.